

《计算机系统基础》

第2章 信息的表示和处理

任课教师:

何璐璐

luluhe@whu.edu.cn

本节内容: 位(bit)、字节、整数和浮点数

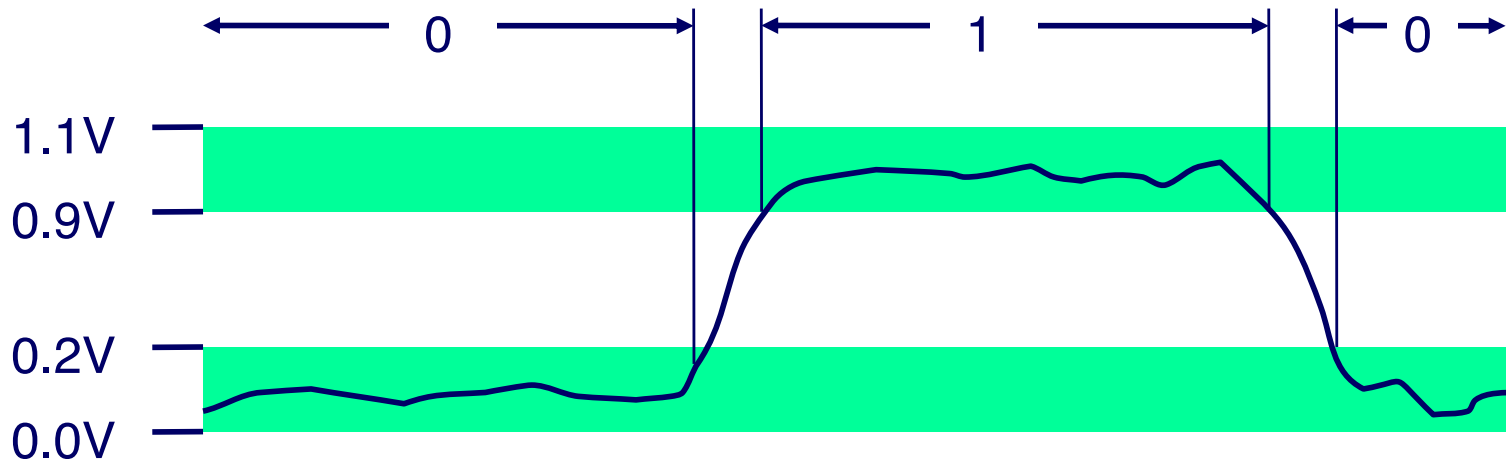
- 用bit来表示信息
- 位级操作
- 整数
 - 表示：无符号数unsigned和有符号数signed
 - 转换，强制类型转换
 - 扩展，截断
 - 加，取负，乘法，移位
 - 小结
- 内存中的数据表示，指针，字符串
- 浮点数

本节内容: 位(bit)、字节、整数和浮点数

- 用bit来表示信息
- 位级操作
- 整数
 - 表示：无符号数unsigned和有符号数signed
 - 转换，强制类型转换
 - 扩展，截断
 - 加，取负，乘法，移位
 - 小结
- 内存中的数据表示，指针，字符串
- 浮点数

凡事皆是bit

- 每个bit为0或1
- 以不同的方式编码/解读bits
 - 计算机确定要做什么(指令)
 - 对数字、集合、字符串等进行表示和处理
- 为什么用bit呢？ 电气特性决定的
 - 存储双状态元素比较容易
 - 在有噪声和误差的线路上也能够可靠地传输



例如，能够以二进制方式计数

■ 基数为2的数的表示

- 430072_{10} 表示为 $110\ 1000\ 1111\ 1111\ 1000_2$
- 1.20_{10} 表示为 $1.0011001100110011[0011]..._2$
- 1.5213×10^4 表示为 $1.1101101101101_2 \times 2^{13}$

对字节值进行编码

- 1字节 (Byte) = 8 bits
 - 二进制: $00000000_2 \sim 11111111_2$
 - 十进制: 0_{10} 到 255_{10}
 - 十六进制: 00_{16} 到 FF_{16}
 - 以16为基的数的表示
 - 使用字符 '0' ~ '9' , 'A' ~ 'F'
 - 在C语言中 将 $FA1D37B_{16}$ 写作 :
 - `0xFA1D37B`
 - `0xfa1d37b`

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

数据表示示例

C Data Type	Typical 32-bit	Typical 64-bit	x86-64
char	1	1	1
short	2	2	2
int	4	4	4
long	4	8	8
float	4	4	4
double	8	8	8
pointer	4	8	8

本节内容: 位(bit)、字节、整数和浮点数

- 用bit来表示信息
- 位级操作
- 整数
 - 表示：无符号数unsigned和有符号数signed
 - 转换，强制类型转换
 - 扩展，截断
 - 加，取负，乘法，移位
 - 小结
- 内存中的数据表示，指针，字符串
- 浮点数

布尔代数

■ George Boole于19世纪提出

■ 逻辑的代数表示

- 将“True”编码为1，“False”编码为0

与 And

- $A \& B = 1$ 当两者 $A=1$ 且 $B=1$

$\&$	0	1
0	0	0
1	0	1

非 Not

- $\sim A = 1$ 当 $A=0$

$\sim$	
0	1
1	0

或 Or

- $A | B = 1$ 当 $A=1$ 或 $B=1$

$ $	0	1
0	0	1
1	1	1

异或 Exclusive-Or (Xor)

- $A \wedge B = 1$ 当 $A=1$ 或 $B=1$, 但不能同时存在

$\wedge$	0	1
0	0	1
1	1	0

通用布尔代数

■ 对位向量进行操作

- 操作是按位进行的

01101001	01101001	01101001	01101001
<u>& 01010101</u>	<u> 01010101</u>	<u>^ 01010101</u>	<u>~ 01010101</u>
01000001	01111101	00111100	10101010

■ 保持布尔代数的所有属性

示例：集合的表示和操作

■ 表示

- 用宽度为 w 位的向量表示 $\{0, \dots, w-1\}$ 的子集

- $a_j = 1$ if $j \in A$

- 01101001 $\{0, 3, 5, 6\}$

- 76543210

- 01010101 $\{0, 2, 4, 6\}$

- 76543210

■ 运算

■ &	交	01000001	$\{0, 6\}$
■	并	01111101	$\{0, 2, 3, 4, 5, 6\}$
■ ^	对称差	00111100	$\{2, 3, 4, 5\}$
■ ~	补	10101010	$\{1, 3, 5, 7\}$

C 语言中的位级运算

- C 语言中有运算 $\&$, $|$, $\sim$, $\wedge$
 - 可应用于任何“整数”数据类型上
 - long, int, short, char, unsigned
 - 将参数看作位向量
 - 对参数按位处理
- 示例 (char 数据类型)
 - $\sim 0x41 \rightarrow 0xBE$
 - $\sim 01000001_2 \rightarrow 10111110_2$
 - $\sim 0x00 \rightarrow 0xFF$
 - $\sim 00000000_2 \rightarrow 11111111_2$
 - $0x69 \& 0x55 \rightarrow 0x41$
 - $01101001_2 \& 01010101_2 \rightarrow 01000001_2$
 - $0x69 | 0x55 \rightarrow 0x7D$
 - $01101001_2 | 01010101_2 \rightarrow 01111101_2$

对比：C 语言中的逻辑运算

■ 与逻辑运算符对比

- &&, ||, !
 - 将 0 视为“False”
 - 任何非0 的值都是“True”
 - 总是返回0或1
 - 提前终止

■ 示例（char数据类型）

- !0x41 → 0x00
- !0x00 → 0x01
- !!0x41 → 0x01

- 0x69 && 0x55 → 0x01
- 0x69 || 0x55 → 0x01
- p && *p (避免访问空指针)
- a && 5/a

要小心 && vs. & (以及|| vs. |)...
超级常见的C语言编程错误!

移位运算

- **左移运算: $x \ll y$**
 - 将位向量 x 左移 y 位
 - 在左边把多余的位丢掉
 - 右边补上0
- **右移运算: $x \gg y$**
 - 将位向量 x 右移 y 位
 - 在左边把多余的位丢掉
 - 逻辑右移
 - 在左边补上0
 - 算数右移
 - 在左边补最高有效位的值
- **未定义行为**
 - 移位数量 < 0 或者 $\geq$ 字长

Argument x	<u>0</u> 1100010
$\ll 3$	000 <u>1</u> 0000
Log. $\gg 2$	0001100 <u>0</u>
Arith. $\gg 2$	<u>00</u> 011000

Argument x	<u>1</u> 0100010
$\ll 3$	000 <u>1</u> 0000
Log. $\gg 2$	0010100 <u>0</u>
Arith. $\gg 2$	<u>11</u> 101000

本节内容: 位(bit)、字节、整数和浮点数

- 用bit来表示信息
- 位级操作
- **整数**
 - 表示：无符号数unsigned和有符号数signed
 - 转换，强制类型转换
 - 扩展，截断
 - 加，取负，乘法，移位
 - 小结
- 内存中的数据表示，指针，字符串
- **浮点数**

整数编码

无符号数 unsigned

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

补码

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

```
short int x = 15213;  
short int y = -15213;
```

符号位

■ C语言中 short 的长度为2字节

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
y	-15213	C4 93	11000100 10010011

■ 符号位

- 对于补码，最高有效位为符号位
 - 0 表示非负
 - 1 表示负数

补码编码示例：

	-16	8	4	2	1	
10 =	0	1	0	1	0	$8 + 2 = 10$

	-16	8	4	2	1	
-10 =	1	0	1	1	0	$-16 + 4 + 2 = -10$

补码编码示例(续)

x = 430072: 0110 1000 1111 1111 1000
y = -430072: 1001 0111 0000 0000 1000

	430072			- 43002		
权重	bit	值	余	bit	值	余
1	0	0	0	0	0	0
2	0	0	0	0	0	0
4	0	0	0	0	0	0
8	1	8	0	1	8	0
16	1	16	8	0	0	8
32	1	32	24	0	0	8
64	1	64	56	0	0	8
128	1	128	120	0	0	8
256	1	256	248	0	0	8
512	1	512	504	0	0	8
1024	1	1024	1016	0	0	8
2048	1	2048	2040	0	0	8
4096	0	0	4088	1	4096	8
8192	0	0	4088	1	8192	4104
16384	0	0	4088	1	16384	12296
32768	1	32768	4088	0	0	28680
65536	0	0	36856	1	65536	28680
131072	1	131072	36856	0	0	94216
262144	1	262144	167928	0	0	94216
-524288	0	0		1	-524288	94216

补码编码示例(练习)

x = 15213:
y = -15213:

权重	15213	-15213
1		
2		
4		
8		
16		
32		
64		
128		
256		
512		
1024		
2048		
4096		
8192		
16384		
-32768		
和	15213	-15213

数值的范围

■ 无符号数值

- $UMin = 0$
000...0
- $UMax = 2^w - 1$
111...1

■ 补码数值

- $TMin = -2^{w-1}$
100...0
- $TMax = 2^{w-1} - 1$
011...1

■ 其他值

- -1
111...1

Values for $W = 16$

	二进制	十六进制	二进制
UMax	65535	FF FF	11111111 11111111
TMax	32767	7F FF	01111111 11111111
TMin	-32768	80 00	10000000 00000000
-1	-1	FF FF	11111111 11111111
0	0	00 00	00000000 00000000

不同字长的特殊值

	W			
	8	16	32	64
UMax	255	65,535	4,294,967,295	18,446,744,073,709,551,615
TMax	127	32,767	2,147,483,647	9,223,372,036,854,775,807
TMin	-128	-32,768	-2,147,483,648	-9,223,372,036,854,775,808

观察到

- $|TMin| = TMax + 1$
 - 非对称的数值范围
- $UMax = 2 * TMax + 1$

■ C 语言编程

- `#include <limits.h>`
- 声明常量，例如
 - `ULONG_MAX`
 - `LONG_MAX`
 - `LONG_MIN`
- 这些值是与平台相关的

无符号数与有符号数的数值

X	B2U(X)	B2T(X)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

■ 等价性

- 非负数值的二进制编码相同

■ 唯一性

- 每个位模式表示不同的整数数值
- 每个可表示整数都有唯一的位编码

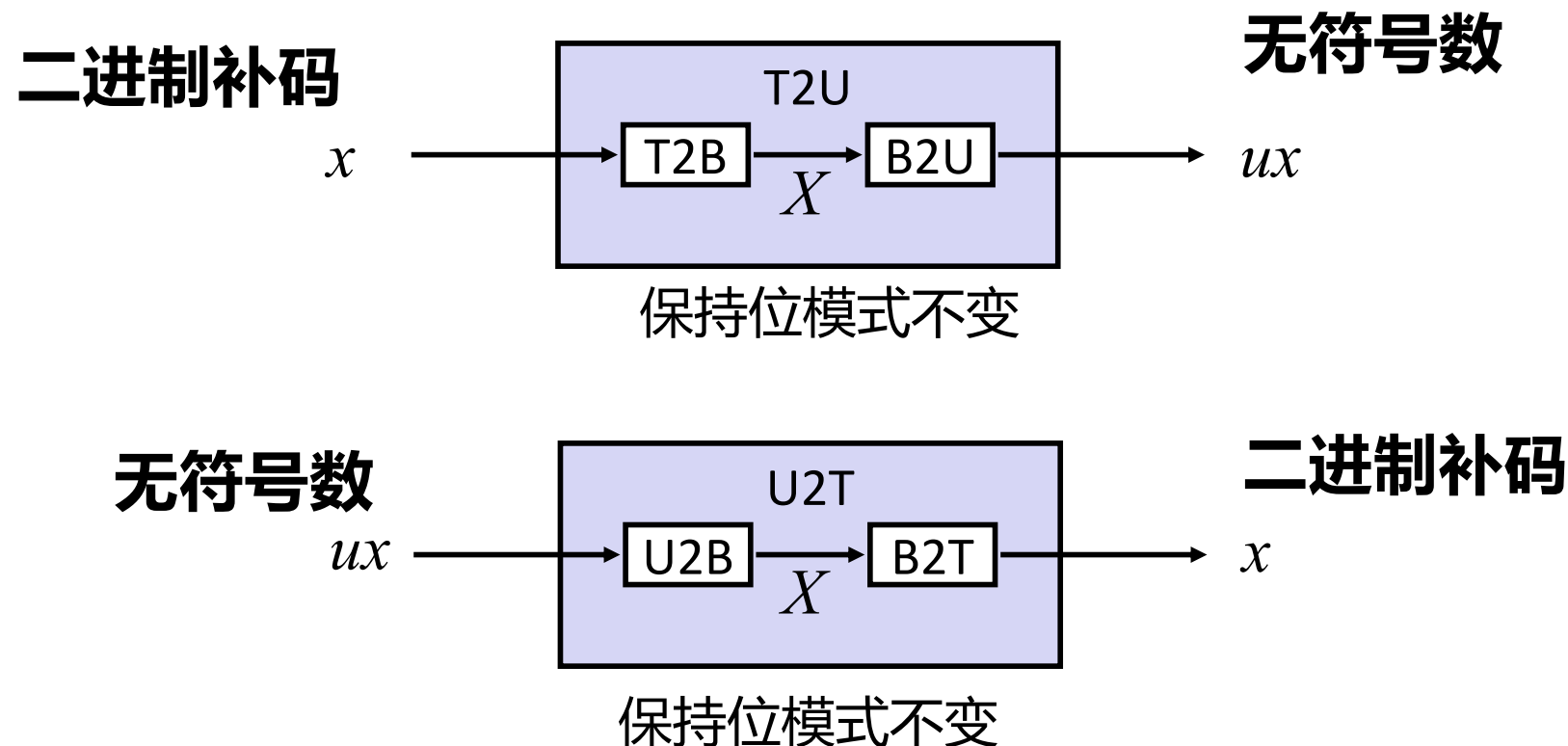
■ $\Rightarrow$ 可以互相转换

- $U2B(x) = B2U^{-1}(x)$
 - 无符号数的位模式
- $T2B(x) = B2T^{-1}(x)$
 - 补码整数的位模式

本节内容: 位(bit)、字节、整数和浮点数

- 用bit来表示信息
- 位级操作
- **整数**
 - 表示：无符号数unsigned和有符号数signed
 - 转换，强制类型转换
 - 扩展，截断
 - 加，取负，乘法，移位
 - 小结
- 内存中的数据表示，指针，字符串
- 浮点数

有符号数和无符号数之间的映射



- 无符号数和补码之间的映射：
保持位模式不变，重新解释

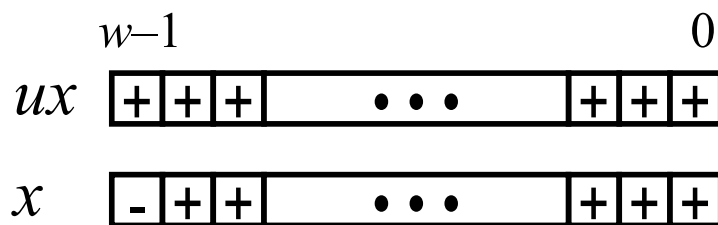
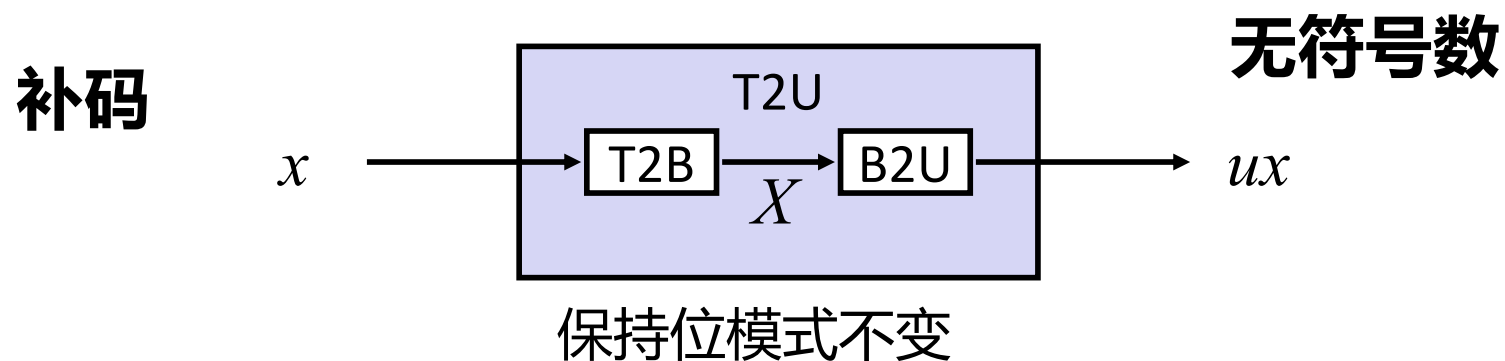
映射关系：有符号数 ↔ 无符号数

位模式	有符号		无符号
0000	0		0
0001	1		1
0010	2		2
0011	3		3
0100	4		4
0101	5	→ T2U →	5
0110	6		6
0111	7	← U2T ←	7
1000	-8		8
1001	-7		9
1010	-6		10
1011	-5		11
1100	-4		12
1101	-3		13
1110	-2		14
1111	-1		15

映射关系：有符号数 ↔ 无符号数

位模式	有符号		无符号
0000	0	=	0
0001	1		1
0010	2		2
0011	3		3
0100	4		4
0101	5		5
0110	6		6
0111	7		7
1000	-8	+/- 16	8
1001	-7		9
1010	-6		10
1011	-5		11
1100	-4		12
1101	-3		13
1110	-2		14
1111	-1		15

有符号数与无符号数之间的关系



大的**负**权重

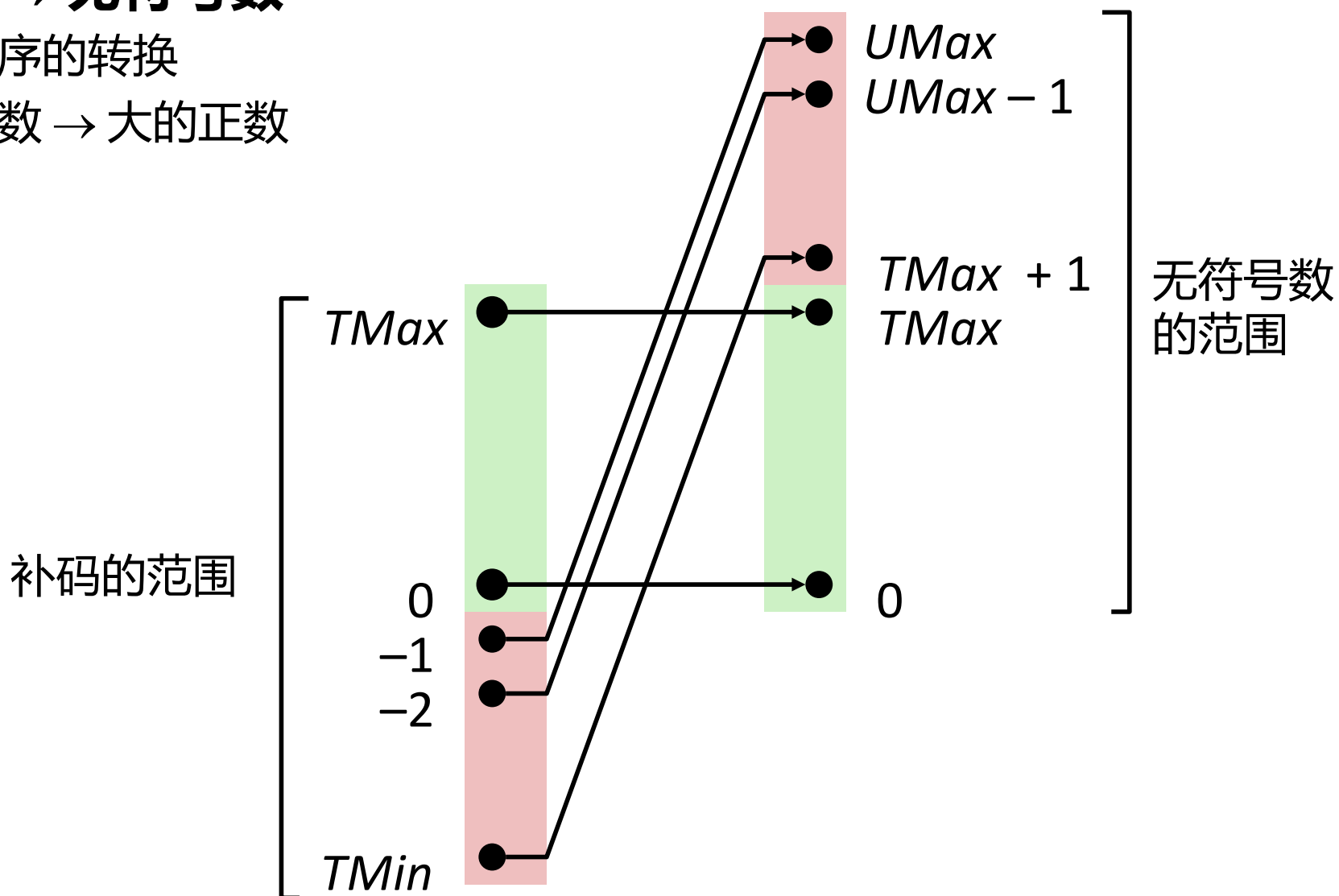
变成

大的**正**权重

转换可视化

■ 补码 → 无符号数

- 有序转换
- 负数 → 大的正数



C语言中的有符号数和无符号数的对比

■ 常量

- 默认情况下被认为是有符号数
- 如果后缀为“U”，则为无符号数
0U, 4294967259U

■ 强制类型转换

- 有符号和无符号之间的明确的强制类型转换跟U2T和T2U一样

```
int tx, ty;  
unsigned ux, uy;  
tx = (int) ux;  
uy = (unsigned) ty;
```

- 赋值和过程调用中的隐式的强制类型转换

```
tx = ux;                                int fun(unsigned u);  
uy = ty;                                uy = fun(tx);
```

强制类型转换中的 surprises!

■ 表达式求值

- 如果单个表达式中混合了有符号数和无符号数，
有符号数被隐式的强制类型转换为无符号数
- 包括比较运算符 <, >, ==, <=, >=
- 示例 $W = 32$: **$TMIN = -2,147,483,648$, $TMAX = 2,147,483,647$**

■ 常数 ₁	常数 ₂	关系运算	求值
0	0U	==	unsigned
-1	0	<	signed
-1	0U	>	unsigned
2147483647	-2147483647-1	>	signed
2147483647U	-2147483647-1	<	unsigned
-1	-2	>	signed
(unsigned)-1	-2	>	unsigned
2147483647	2147483648U	<	unsigned
2147483647	(int) 2147483648U	>	signed

无符号数 vs 有符号数：更容易犯错

```
unsigned i;  
for (i = cnt-2; i >= 0; i--)  
    a[i] += a[i+1];
```

- 代码可以更加巧妙一些

```
#define DELTA sizeof(int)  
int i;  
for (i = CNT; i-DELTA >= 0; i-= DELTA)  
    . . .
```

总结

有符号数 $\leftrightarrow$ 无符号数: 基本的转换规则

- 位模式保持不变
- 但是重新解释
- 可能会有意想不到的结果: 加/减 2^w
- 表达式中同时包含有符号和无符号整数
 - 有符号整数 `int` 被转换为无符号数 `unsigned` ! !
- $|TMAX| = |TMIN| - 1$
 - 如果 $x = TMIN$ 呢 ?

```
if (x < 0)
    return -x;
else
    return x;
```
- $UMAX = 2 TMAX + 1$

代码安全示例#1

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

- FreeBSD 中 `getpeername` 方法的代码实现简化版本
- 存在安全漏洞

典型应用

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

```
#define MSIZE 528

void getstuff() {
    char mybuf[MSIZE];
    copy_from_kernel(mybuf, MSIZE);
    printf("%s\n", mybuf);
}
```

恶意应用

```
/* Declaration of library function memcpy */  
void *memcpy(void *dest, void *src, size_t n);
```

```
/* Kernel memory region holding user-accessible data */  
#define KSIZE 1024  
char kbuf[KSIZE];  
  
/* Copy at most maxlen bytes from kernel region to user buffer */  
int copy_from_kernel(void *user_dest, int maxlen) {  
    /* Byte count len is minimum of buffer size and maxlen */  
    int len = KSIZE < maxlen ? KSIZE : maxlen;  
    memcpy(user_dest, kbuf, len);  
    return len;  
}
```

```
#define MSIZE 528  
  
void getstuff() {  
    char mybuf[MSIZE];  
    copy_from_kernel(mybuf, -MSIZE);  
    . . .  
}
```

本节内容: 位(bit)、字节、整数和浮点数

- 用bit来表示信息
- 位级操作
- **整数**
 - 表示：无符号数unsigned和有符号数signed
 - 转换，强制类型转换
 - 扩展，截断
 - 加，取负，乘法，移位
 - 小结
- 内存中的数据表示，指针，字符串
- 浮点数

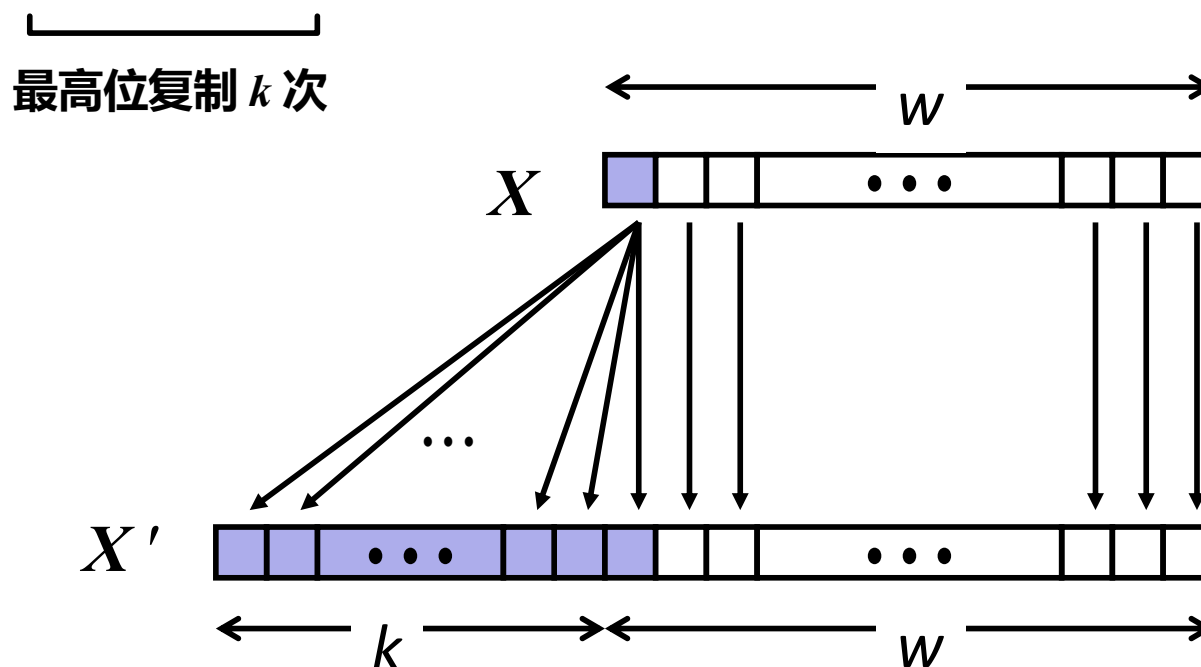
符号扩展

■ 任务：

- 给定 w 位有符号整数 X
- 将其转换为数值相等的 $w+k$ 位整数 X'

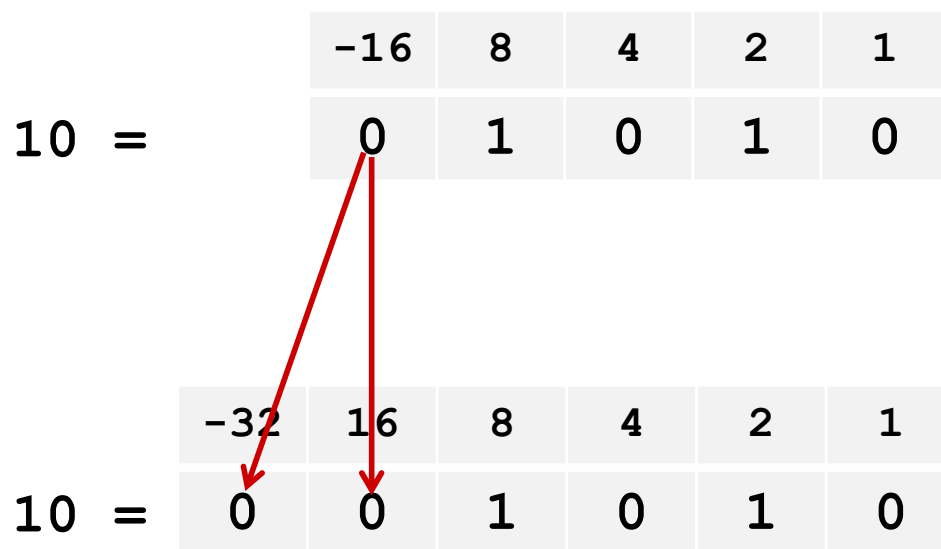
■ 规则：

- 符号位复制 k 次:
- $X' = \underbrace{X_{w-1}, \dots, X_{w-1}}_{\text{最高位复制 } k \text{ 次}}, X_{w-1}, X_{w-2}, \dots, X_0$

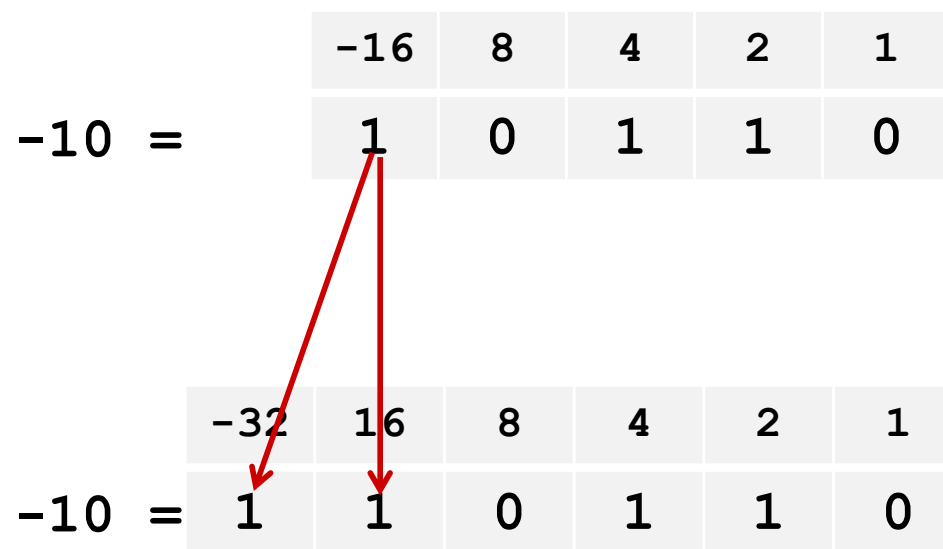


符号扩展：简单示例

正数



负数



向更大的符号位扩展的示例

```
short int x = 15213;  
int      ix = (int) x;  
short int y = -15213;  
int      iy = (int) y;
```

	小数	十六进制	二进制
x	15213	3B 6D	00111011 01101101
ix	15213	00 00 3B 6D	00000000 00000000 00111011 01101101
y	-15213	C4 93	11000100 10010011
iy	-15213	FF FF C4 93	11111111 11111111 11000100 10010011

- 将较小的整数数据类型转换成较大的整数数据类型
- C语言自动进行符号扩展

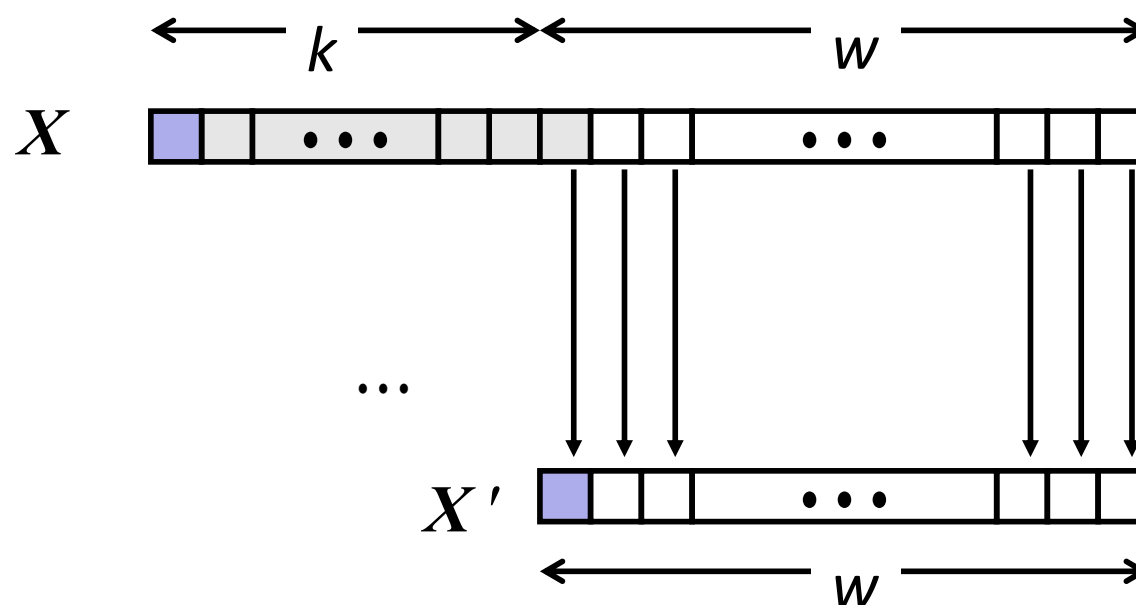
截断

■ 任务：

- 给定 $k+w$ 位有符号或无符号整数 X
- 将其转换为 w 位整数 X' ，对于“足够小”的 X 具有相同的值

■ 规则：

- 最高位减少 k 位
- $X' = X_{w-1}, X_{w-2}, \dots, X_0$



截断：示例

符号位没有改变

	-16	8	4	2	1
2 =	0	0	0	1	0

	-8	4	2	1
2 =	0	0	1	0

$$2 \bmod 16 = 2$$

	-16	8	4	2	1
-6 =	1	1	0	1	0

	-8	4	2	1
-6 =	1	0	1	0

$$-6 \bmod 16 = 26\text{U} \bmod 16 = 10\text{U} = -6$$

符号位改变

	-16	8	4	2	1
10 =	0	1	0	1	0

	-8	4	2	1
-6 =	1	0	1	0

$$10 \bmod 16 = 10\text{U} \bmod 16 = 10\text{U} = -6$$

	-16	8	4	2	1
-10 =	1	0	1	1	0

	-8	4	2	1
6 =	0	1	1	0

$$-10 \bmod 16 = 22\text{U} \bmod 16 = 6\text{U} = 6$$

总结：

扩展，截断：基本规则

- **扩展（例如，short 扩展到 int）**
 - 无符号数unsigned：0拓展，在最前面加0
 - 有符号数signed：符号位拓展
 - 两种数据类型都得到与预期相符的结果
- **截断（例如，unsigned 转换到 unsigned short）**
 - 无符号/有符号：高位被截断
 - 结果被重新解释
 - 无符号：等价于模(mod)运算
 - 有符号数：类似模(mod)运算
 - 对于较小的数字产生预期的结果

本节内容: 位(bit)、字节、整数和浮点数

- 用bit来表示信息
- 位级操作
- **整数**
 - 表示：无符号数unsigned和有符号数signed
 - 转换，强制类型转换
 - 扩展，截断
 - 加，取负，乘法，移位
 - 小结
- 内存中的数据表示，指针，字符串
- 浮点数

无符号数加法

操作数： w 位

u

+ v

真实的和： $w+1$ 位

$u + v$

丢弃进位： w 位

$\text{UAdd}_w(u, v)$

■ 标准的加法函数

- 忽略进位输出

■ 实现模运算

$$s = \text{UAdd}_w(u, v) = u + v \bmod 2^w$$

unsigned char

```

      1110 1001
    + 1101 0101
    -----
  1 1011 1110
    -----
      1011 1110
  
```

```

      E9
    + D5
    -----
    1BE
    -----
      BE
  
```

```

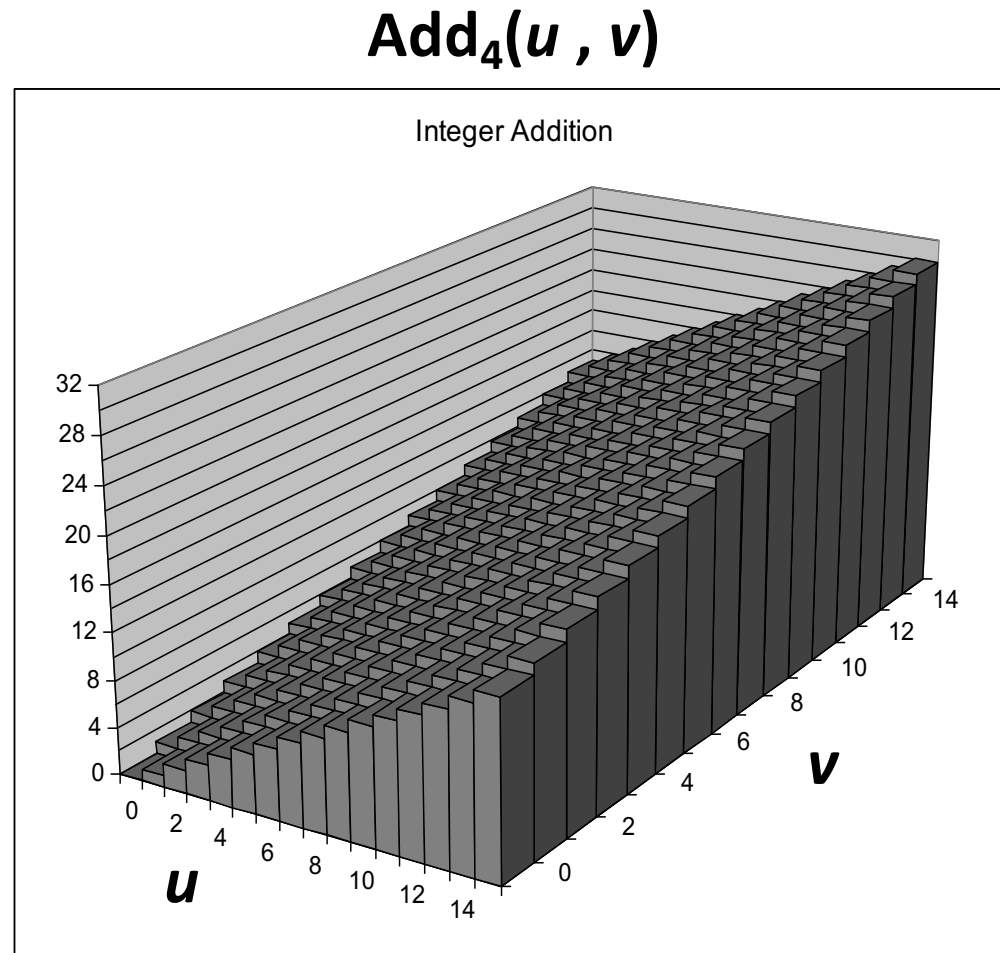
      233
    + 213
    -----
    446
    -----
      190
  
```

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

整数加法(数学上)的可视化

■ 整数加法

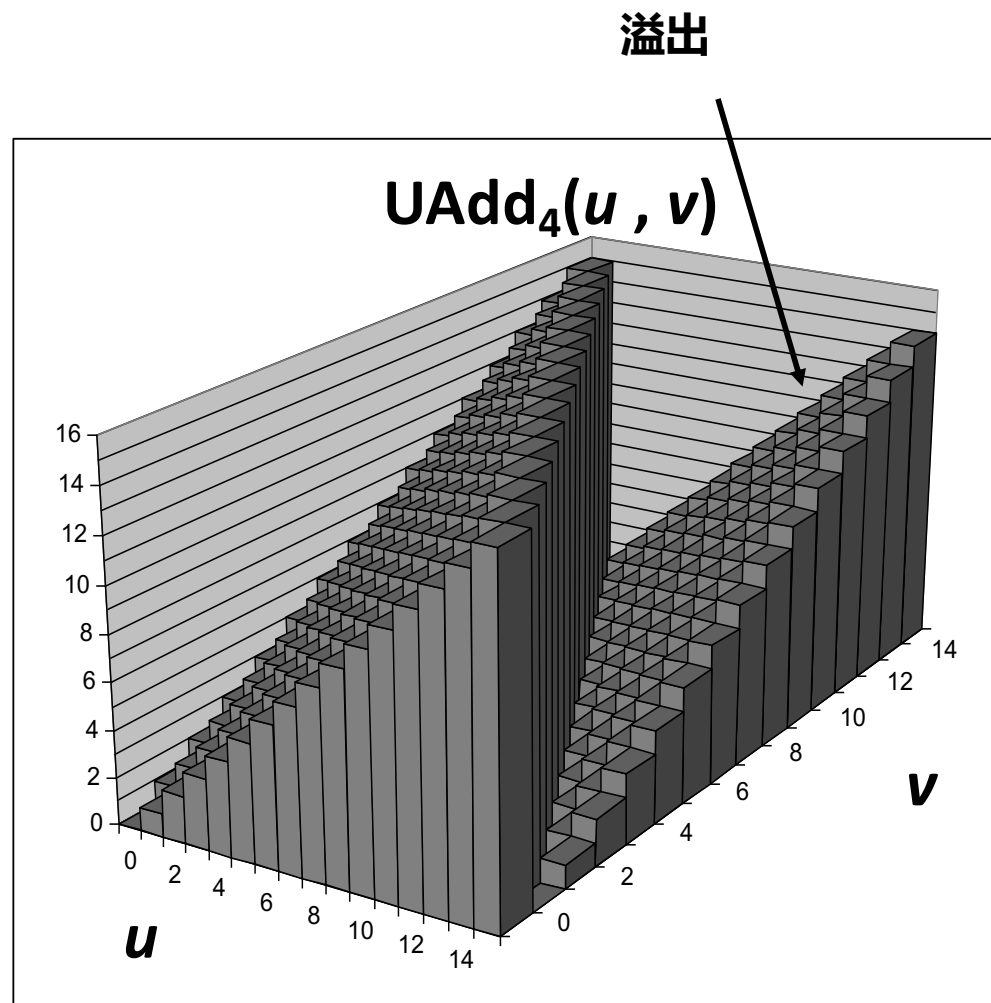
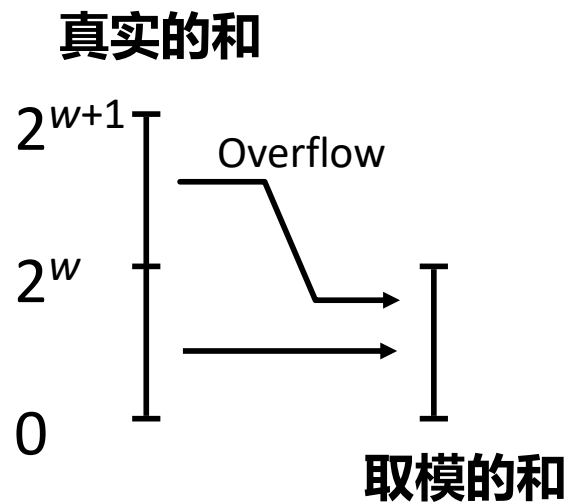
- 4位整数 u, v
- 计算真实的和 $\text{Add}_4(u, v)$
- 值随 u 和 v 线性增加
- 形成平坦曲面



无符号加法的可视化

■ 环绕

- 如果真实的和 $\geq 2^w$
- 最多一次



应用示例：检测UAdd溢出

```
/* 判断无符号数x和y相加是否会产生溢出。如果不会，返回1；否则返回0。*/  
int uadd_ok(unsigned x, unsigned y) {  
    unsigned sum = x+y;  
    return sum >= x;  
}
```

补码加法

unsigned char

$$\begin{array}{r} 1110\ 1001 \\ +\ 1101\ 0101 \\ \hline 1\ 1011\ 1110 \\ \hline 1011\ 1110 \end{array}$$
$$\begin{array}{r} E9 \\ +\ D5 \\ \hline 1BE \\ \hline BE \end{array}$$
$$\begin{array}{r} 233 \\ +\ 213 \\ \hline 446 \\ \hline 190 \end{array}$$

■ TAdd 和 UAdd 具有相同的位级行为

- C语言中的有符号数加法和无符号数加法：

```
int s, t, u, v;
```

```
s = (int) ((unsigned) u + (unsigned) v);
```

```
t = u + v
```

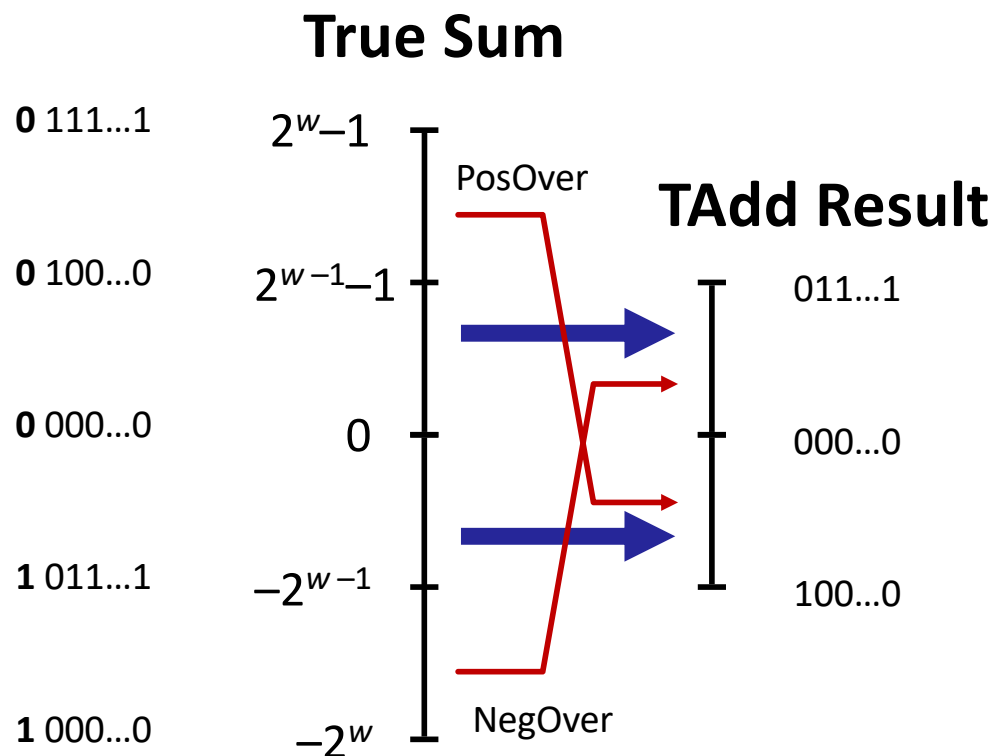
- `s == t` ? ?

$$\begin{array}{r} 1110\ 1001 \\ +\ 1101\ 0101 \\ \hline 1\ 1011\ 1110 \\ \hline 1011\ 1110 \end{array}$$
$$\begin{array}{r} E9 \\ +\ D5 \\ \hline 1BE \\ \hline BE \end{array}$$
$$\begin{array}{r} -23 \\ +\ -43 \\ \hline -66 \\ \hline -66 \end{array}$$

补码加法的溢出

■ 功能

- 真和需要 $w+1$ 位
- 丢弃 MSB
- 将剩余位视为二进制补码整数



补码加法的可视化

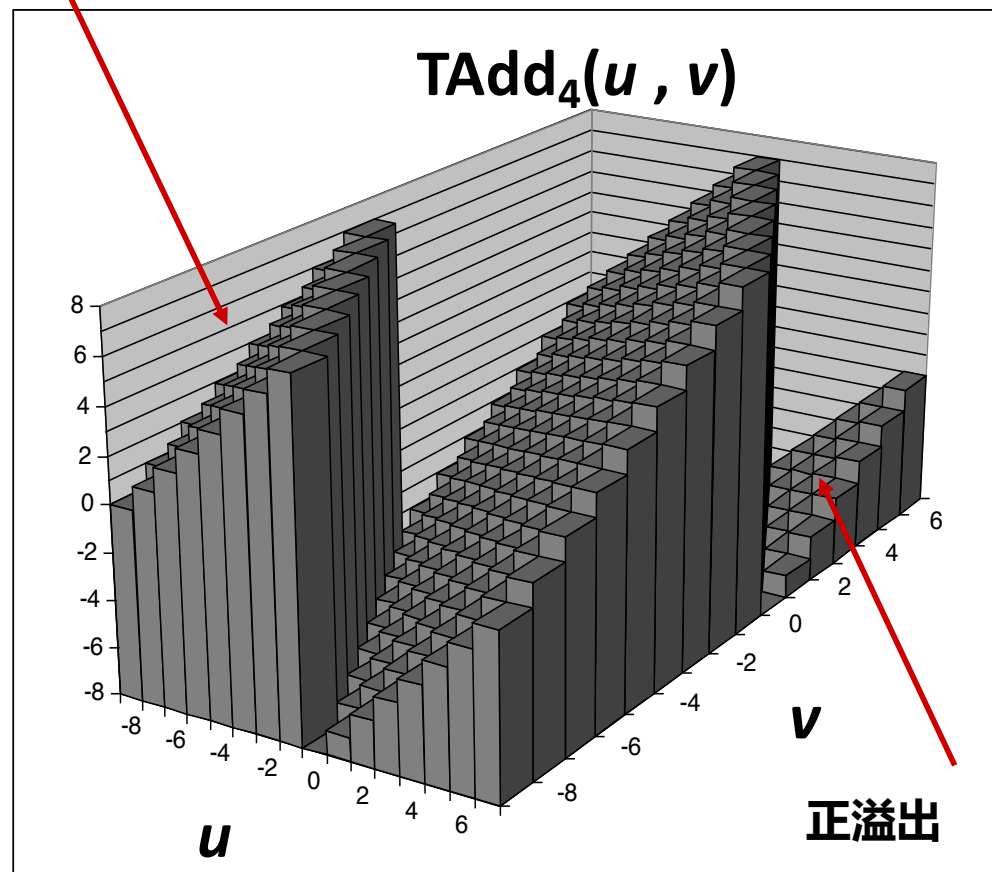
■ 取值

- 4位二进制补码
- 取值范围是 -8 到 +7

■ 环绕

- If $\text{sum} \geq 2^{w-1}$
 - 变为负数
 - 至多一次
- If $\text{sum} < -2^{w-1}$
 - 变为正数
 - 至多一次

负溢出



检测TAdd溢出 – 方法1

/* 判断有符号数x和y相加是否会产生溢出。如果不会，返回1；否则返回0。*/

```
int tadd_ok(int x, int y) {
```

```
}
```

检测TAdd溢出 – 方法1

/* 判断有符号数x和y相加是否会产生溢出。如果不会，返回1；否则返回0。*/

```
int tadd_ok(int x, int y) {  
    int sum = x+y;  
    int neg_over = x < 0 && y < 0 && sum >= 0;  
    int pos_over = x >= 0 && y >= 0 && sum < 0;  
    return !neg_over && !pos_over;  
}
```

- For x and y in the range $TMin_w \leq x, y \leq TMax_w$, let $s = x +_w^t y$. Then the computation of s has had positive overflow if and only if $x > 0$ and $y > 0$ but $s \leq 0$. The computation has had negative overflow if and only if $x < 0$ and $y < 0$ but $s \geq 0$.

检测TAdd溢出 – 方法2

/* 判断有符号数x和y相加是否会产生溢出。如果不会，返回1；否则返回0。*/

```
int tadd_ok(int x, int y) {  
    int sum = x+y;  
    return (sum-x == y) && (sum-y == x);  
}
```

检测TAdd溢出 – 方法2

```
/* 判断有符号数x和y相加是否会产生溢出。如果不会，返回1；否则返回0。*/  
  
/* WARNING: This code is buggy. */  
int tadd_ok(int x, int y) {  
    int sum = x+y;  
    return (sum-x == y) && (sum-y == x);  
}
```

检测TAdd溢出 – 方法3

/* 判断有符号数x和y相加是否会产生溢出。如果不会，返回1；否则返回0。*/

```
int tsub_ok(int x, int y) {  
    return tadd_ok(x, -y);  
}
```

检测TAdd溢出 – 方法3

```
/* 判断有符号数x和y相加是否会产生溢出。如果不会，返回1；否则返回0。*/
```

```
/* WARNING: This code is buggy. */
```

```
int tsub_ok(int x, int y) {  
    return tadd_ok(x, -y);  
}
```


乘法

■ 目标：计算 w 位数 x 和 y 的乘积

- 有符号数signed，也可以是无符号数unsigned

■ 但是精确的结果可能大于 w 位

- 无符号数：最高可为 $2w$ 位
 - 结果范围： $0 \leq x * y \leq (2^w - 1)^2 = 2^{2w} - 2^{w+1} + 1$
- 补码最小值(负数)：最高可为 $2w-1$ 位
 - 结果范围： $x * y \geq (-2^{w-1}) * (2^{w-1} - 1) = -2^{2w-2} + 2^{w-1}$
- 补码最大值(正数)：最高可为 $2w$ 位，仅适用于 $(TMin_w)^2$
 - 结果范围： $x * y \leq (-2^{w-1})^2 = 2^{2w-2}$

■ 所以，要保持精确的结果...

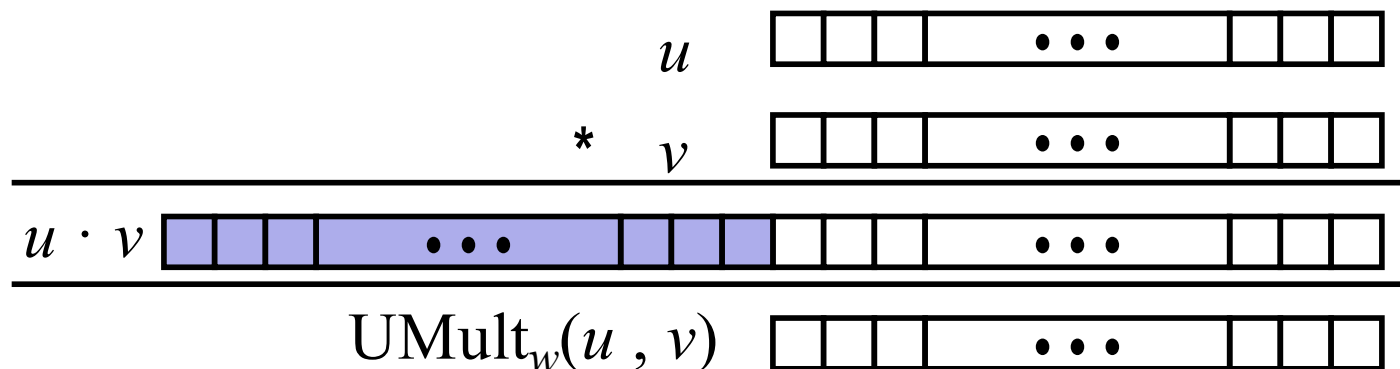
- 需要在计算乘积时，扩展表示乘积结果的字节数
- 如果需要，可以用软件完成
 - 例如，通过“任意精度arbitrary precision”算术包

C语言中的无符号数乘法

操作数： w 位

真实的乘积： $2*w$ 位

丢弃 w 位： w bits



■ 标准乘法功能

- 忽略高 w 位

■ 实现模运算

- $\text{UMult}_w(u, v) = u \cdot v \bmod 2^w$

$$\begin{array}{r}
 \\
 \\
 \\
 \hline
 1100 \ 0001 \ 1101 \ 1101 \\
 \hline
 \\
 \\
 \\
 \hline
 1101 \ 1101
 \end{array}$$

$$\begin{array}{r}
 \\
 \\
 \\
 \hline
 C1DD \\
 \hline
 \\
 \\
 \\
 \hline
 DD
 \end{array}$$

$$\begin{array}{r}
 \\
 \\
 \\
 \hline
 49629 \\
 \hline
 \\
 \\
 \\
 \hline
 221
 \end{array}$$

C语言中的有符号数乘法

■ 标准乘法功能

- 忽略高w位
- 有符号乘法和无符号乘法有些结果不同
- 低位是一样的

unsigned	*	1110 1001	E9	233
		1101 0101	D5	213
		1100 0001 1101 1101	C1DD	49629
		1101 1101	DD	221
signed	*	1110 1001	E9	-23
		1101 0101	D5	-43
		0000 0011 1101 1101	03DD	989
		1101 1101	DD	-35

3位无符号/有符号数乘法示例

Mode	x	y	$x \cdot y$	Truncated $x \cdot y$
Unsigned	5 [101]	3 [011]	15 [001111]	7 [111]

3位无符号/有符号数乘法示例

Mode	x		y		$x \cdot y$		Truncated $x \cdot y$	
Unsigned	5	[101]	3	[011]	15	[001111]	7	[111]
Two's complement	-3	[101]	3	[011]	-9	[110111]	-1	[111]
Unsigned	4	[100]	7	[111]	28	[011100]	4	[100]
Two's complement	-4	[100]	-1	[111]	4	[000100]	-4	[100]
Unsigned	3	[011]	3	[011]	9	[001001]	1	[001]
Two's complement	3	[011]	3	[011]	9	[001001]	1	[001]

检测TMult溢出（课后练习2.35——证明）

/* 判断有符号数x和y相乘是否会产生溢出。如果不会，返回1；否则返回0。*/

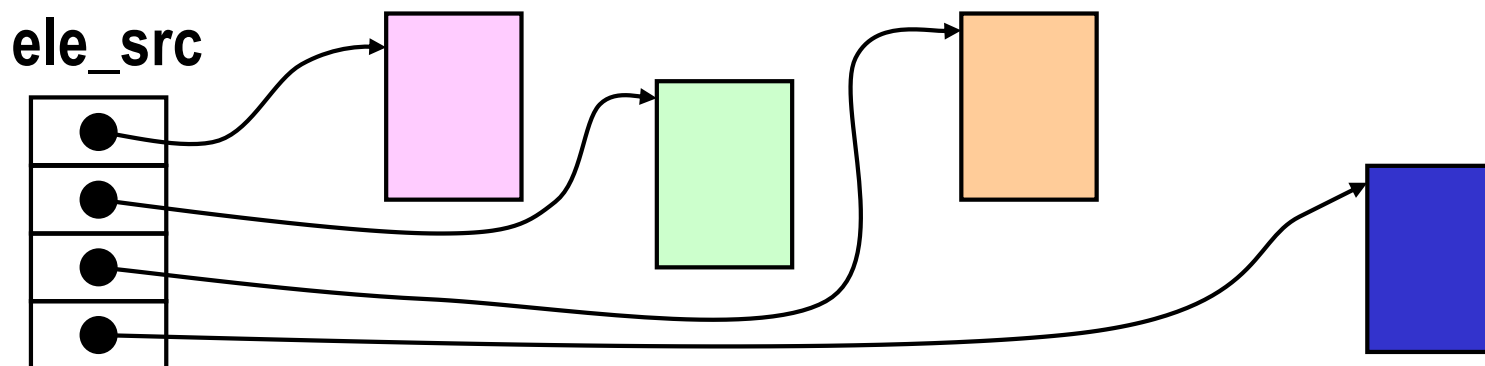
```
int tmult_ok(int x, int y) {  
    int p = x*y;  
    /* Either x is zero, or dividing p by x gives y */  
    return !x || p/x == y;  
}
```

代码安全示例 #2

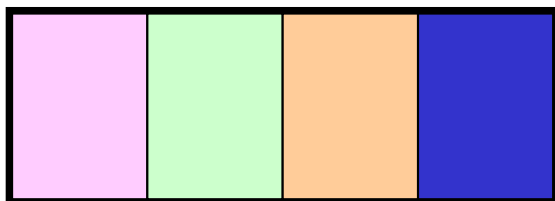
■ SUN XDR library

- XDR库是一个广泛使用的在程序间共享数据的工具

```
void* copy_elements(void *ele_src[], int ele_cnt, size_t ele_size);
```



`malloc(ele_cnt * ele_size)`



XDR Code

```
void *malloc(size_t size);
```

```
void* copy_elements(void *ele_src[], int ele_cnt, size_t ele_size) {  
    /*  
     * Allocate buffer for ele_cnt objects, each of ele_size bytes  
     * and copy from locations designated by ele_src  
     */  
    void *result = malloc(ele_cnt * ele_size);  
    if (result == NULL)  
        /* malloc failed */  
        return NULL;  
    void *next = result;  
    int i;  
    for (i = 0; i < ele_cnt; i++) {  
        /* Copy object i to destination */  
        memcpy(next, ele_src[i], ele_size);  
        /* Move pointer to next memory region */  
        next += ele_size;  
    }  
    return result;  
}
```


带移位的二次幂乘

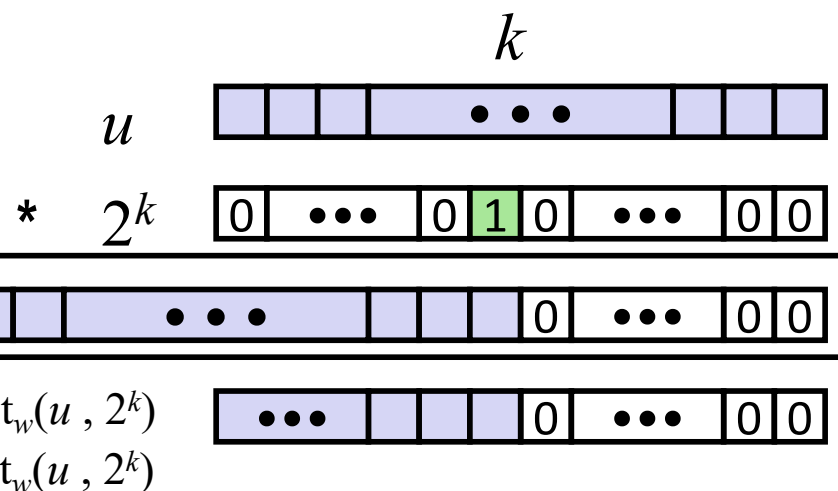
■ 运算

- $u \ll k$ 等价于 $u * 2^k$
- 有符号数或无符号数都是这样

操作数： w 位

真实的结果： $w+k$ 位

丢弃 k 位： w 位



■ 示例

- $u \ll 3 \quad == \quad u * 8$
- $(u \ll 5) - (u \ll 3) \quad == \quad u * 24$
- 大多数机器的移位和加法比乘法快
 - 编译器自动生成此代码

带移位的二次幂乘

■ $x * K$

- $K = [(0...0) (1...1) (0...0) \dots (1...1)]$
- e.g. $14 = [(0...0) (111)(0)], n=3, m=1$

■ A: $(x \ll n) + (x \ll (n-1)) + \dots + (x \ll m)$

- $x * 14 \rightarrow (x \ll 3) + (x \ll 2) + (x \ll 1)$

■ B: $(x \ll (n+1)) - (x \ll m)$

- $x * 14 \rightarrow (x \ll 4) - (x \ll 1)$

编译后的乘法语句

C Function

```
long mul12(long x)
{
    return x*12;
}
```

Compiled Arithmetic Operations

```
leaq (%rax,%rax,2), %rax
salq $2, %rax
```

Explanation

```
t <- x+x*2
return t << 2;
```

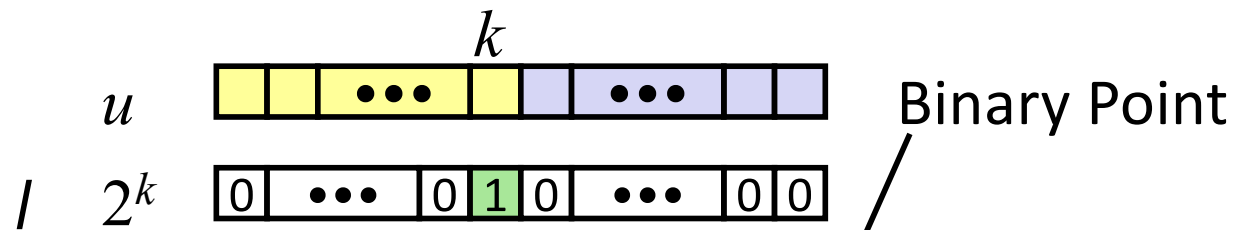
- C compiler automatically generates shift/add code when multiplying by constant

带移位的无符号数2次幂除法

■ 无符号数2的幂的商

- $u \gg k$ 等价于 $\lfloor u / 2^k \rfloor$
- 使用逻辑移位

操作数:



除法：

结果：

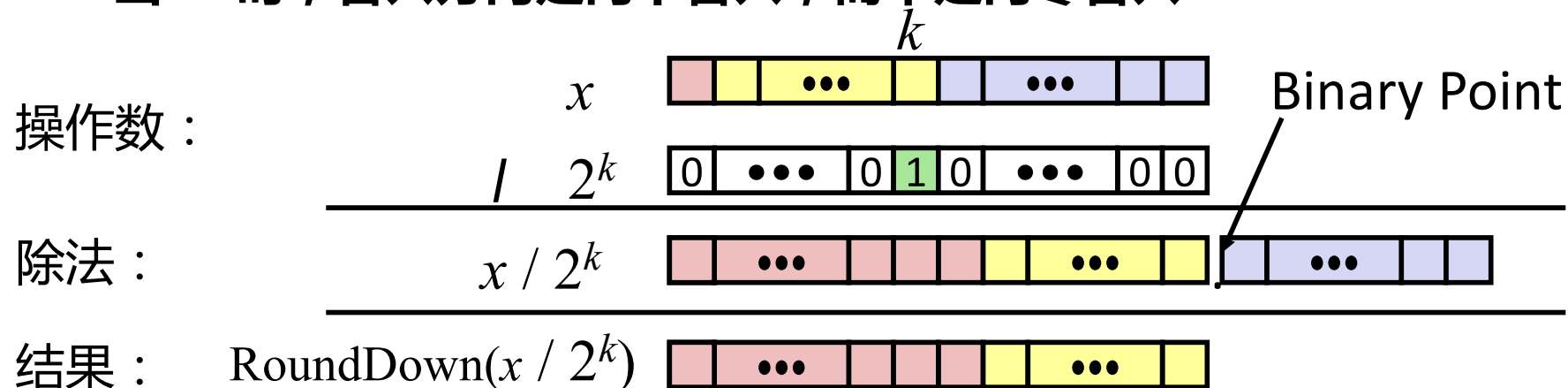
$$\lfloor u / 2^k \rfloor$$

	Division	Computed	Hex	Binary
x	15213	15213	3B 6D	00111011 01101101
x >> 1	7606.5	7606	1D B6	00011101 10110110
x >> 4	950.8125	950	03 B6	00000011 10110110
x >> 8	59.4257813	59	00 3B	00000000 00111011

带移位的有符号2次幂除法

■ 有符号数2的幂的商

- $x \gg k$ 等价于 $\lfloor x / 2^k \rfloor$
- 使用算术移位
- 当 $x < 0$ 时，舍入方向是向下舍入，而不是向零舍入



	Division	Computed	Hex	Binary
x	-15213	-15213	C4 93	11000100 10010011
$x \gg 1$	-7606.5	-7607	E2 49	11100010 01001001
$x \gg 4$	-950.8125	-951	FC 49	11111100 01001001
$x \gg 8$	-59.4257813	-60	FF C4	11111111 11000100

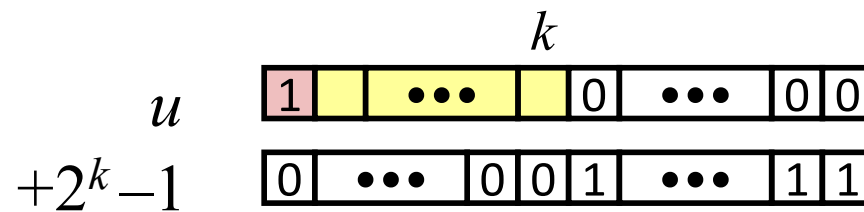
带偏置量的二次幂除法

■ 负数除以 2 的幂的商

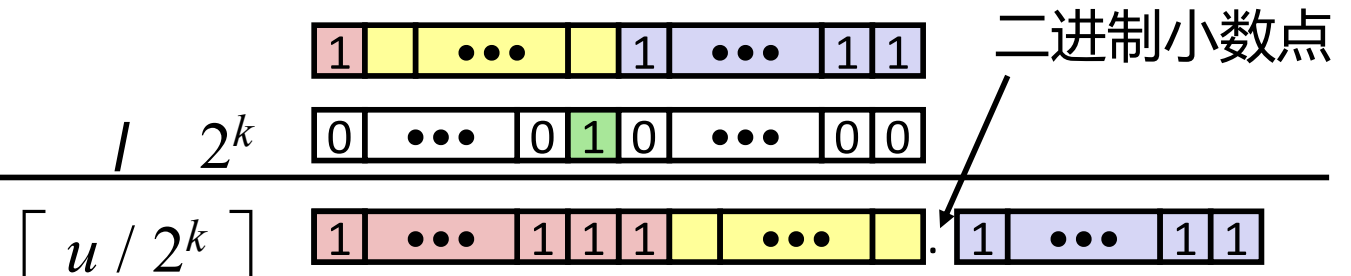
- 欲计算 $\lceil x / 2^k \rceil$ (向 0 舍入)
- 计算为 $\lfloor (x+2^k-1) / 2^k \rfloor$
 - C语言中: $(x + (1 \ll k) - 1) \gg k$
 - 将被除数偏置于 0

Case 1: 没有舍入

被除数:



除数:

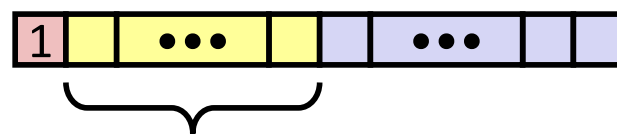
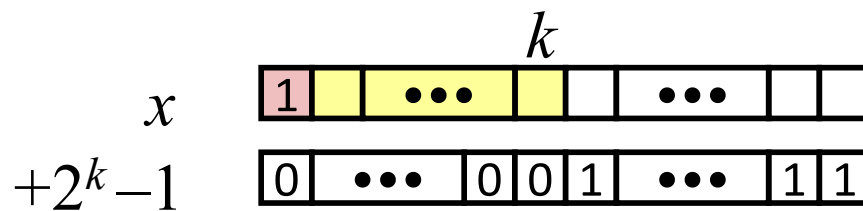


偏置量没有影响

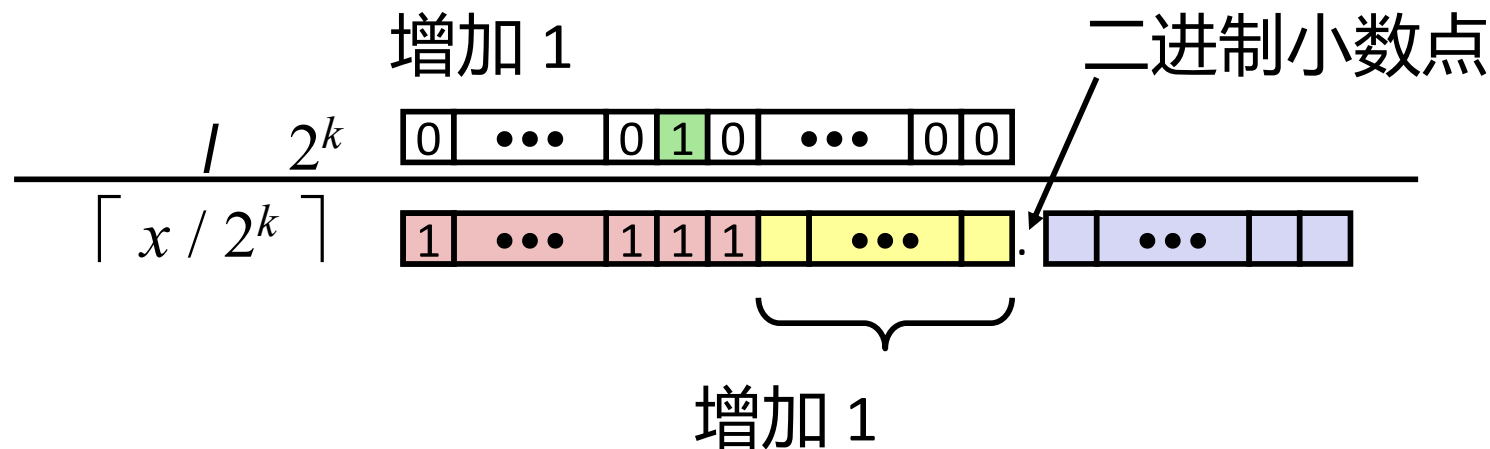
带偏置量的二次幂除法(Cont.)

Case 2: 舍入

被除数：



除数：



偏置量为最终结果+1

无符号数求反

■ 模数加法(Modular Addition)形成一个阿贝尔群(*Abelian Group*)

■ 封闭Closed

$$0 \leq \text{UAdd}_w(u, v) \leq 2^w - 1$$

■ 交换律Commutative

$$\text{UAdd}_w(u, v) = \text{UAdd}_w(v, u)$$

■ 结合律Associative

$$\text{UAdd}_w(t, \text{UAdd}_w(u, v)) = \text{UAdd}_w(\text{UAdd}_w(t, u), v)$$

■ 0 is 加法单位元 additive identity

$$\text{UAdd}_w(u, 0) = u$$

■ 每个元素都有加法的逆元 additive **inverse**

- Let $\text{UComp}_w(u) = 2^w - u$ (**无符号数求反**, 教材p62)
 $\text{UAdd}_w(u, \text{UComp}_w(u)) = 0$

无符号数求反

我们能用一个十六进制数字来表示长度 $\omega=4$ 的位模式。对于这些数字的无符号解释，给出所示数字的无符号加法逆元的位表示（用十六进制形式）

$$(-\frac{u}{4} x) + x = 16$$

x		$-\frac{u}{4} x$	
Hex	Decimal	Decimal	Hex
1			
4			
7			
A			
E			

补码的非

$$-^t_w x = \begin{cases} TMin_w, & x = TMin_w \\ -x, & x > TMin_w \end{cases}$$

- 对 ω 位的补码加法来说， $TMin_\omega$ 是自己的加法的逆，而对其他任何数值 x 都，有 $-x$ 作为其加法的逆。

x		$\sim x$		$\sim x + 1$	
0101	5	1010	-6	1011	-5
0111	7	1000	-8	1001	-7
1100	-4	0011	3	0100	4
0000	0	1111	-1	0000	0
1000	8	0111	7	1000	-8

补码非：求补 & 加 1

- 通过每一位求补和加1来计算取反

$$\sim x + 1 == -x$$

- 示例

- 观察: $\sim x + x == 1111\dots111 == -1$

$$\begin{array}{r} x \quad 10011101 \\ + \quad \sim x \quad 01100010 \\ \hline -1 \quad 11111111 \end{array}$$

x = 15213

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
~x	-15214	C4 92	11000100 10010010
~x+1	-15213	C4 93	11000100 10010011
y	-15213	C4 93	11000100 10010011

补码非 示例

$x = 0$

	Decimal	Hex	Binary
0	0	00 00	00000000 00000000
~ 0	-1	FF FF	11111111 11111111
$\sim 0 + 1$	0	00 00	00000000 00000000

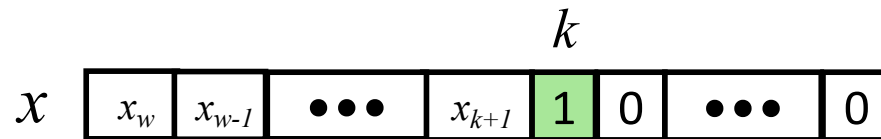
$x = \text{TMin}$

	Decimal	Hex	Binary
x	-32768	80 00	10000000 00000000
$\sim x$	32767	7F FF	01111111 11111111
$\sim x + 1$	-32768	80 00	10000000 00000000

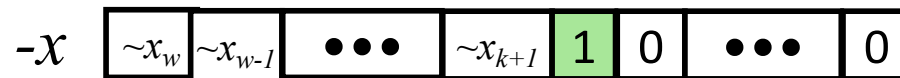
规范反例

补码非：对最右边的1左边所有位取反

- 假设 k 是最右边1的位置



- 对位 k 左边的所有位取反



- 示例

$x = 15213$

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
$-x$	-15213	C4 93	11000100 10010011

本节内容: 位(bit)、字节、整数和浮点数

- 用bit来表示信息
- 位级操作
- **整数**
 - 表示：无符号数unsigned和有符号数signed
 - 转换，强制类型转换
 - 扩展，截断
 - 加，取负，乘法，移位
 - 小结
- 内存中的数据表示，指针，字符串
- 浮点数

算术:基本规则

■ 加法：

- 无符号数/有符号数：普通加法后多余的位需截断，位级操作相同
- 无符号数：加法 $\text{mod } 2^w$
 - 算术加法结果 + 可能减去 2^w
- 有符号数：改性加法 $\text{mod } 2^w$ （结果在适当的范围内）
 - 算术加法结果 + 可能加上或减去 2^w

■ 乘法：

- 无符号数/有符号数：普通乘法后多余的位需截断，位级操作相同
- 无符号数：乘法 $\text{mod } 2^w$
- 有符号数：改性乘法 $\text{mod } 2^w$ （结果在适当的范围内）

为什么要使用无符号数？

- 不要在没有理解含义的情况下使用

- 很容易犯错

```
unsigned i;  
for (i = cnt-2; i >= 0; i--)  
    a[i] += a[i+1];
```

- 可以很巧妙

```
#define DELTA sizeof(int)  
int i;  
for (i = CNT; i-DELTA >= 0; i-= DELTA)  
    . . .
```


用无符号数倒计数

■ 使用无符号数作为循环索引的正确方法

```
unsigned i;  
for (i = cnt-2; i < cnt; i--)  
    a[i] += a[i+1];
```

■ 参考 Robert Seacord, *Secure Coding in C and C++*

- C 标准保证无符号加法的行为类似于模运算
 - $0 - 1 \rightarrow UMax$

■ 甚至更好

```
size_t i;  
for (i = cnt-2; i < cnt; i--)  
    a[i] += a[i+1];
```

- 数据类型 `size_t` 定义为长度 = 字大小的无符号值
- 即使 `cnt = UMax` 代码也能工作
- 如果 `cnt` 是有符号数且 < 0 怎么办？

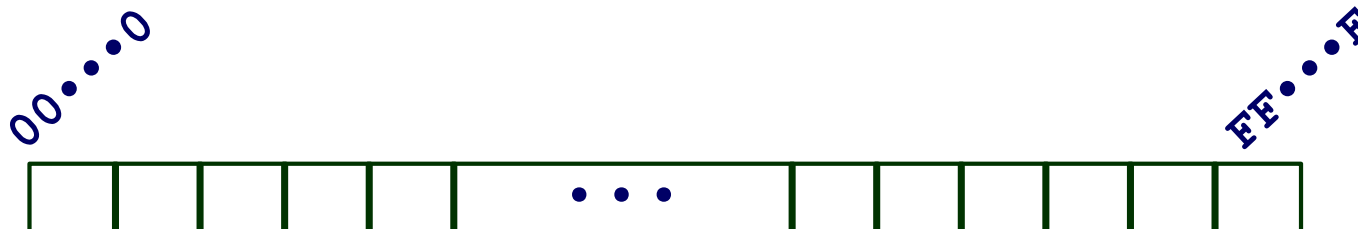
为什么要使用无符号数？(续.)

- **在执行模运算时使用**
 - 多精度运算
- **使用位表示集合时使用**
 - 逻辑右移，没有符号位的拓展
- **在系统级编程中使用**
 - 位掩码、设备命令、...

本节内容: 位(bit)、字节、整数和浮点数

- 用bit来表示信息
- 位级操作
- 整数
 - 表示：无符号数unsigned和有符号数signed
 - 转换，强制类型转换
 - 扩展，截断
 - 加，取负，乘法，移位
 - 小结
- 内存中的数据表示，指针，字符串
- 浮点数

面向字节的内存组织



■ 程序通过地址引用数据

- 从概念上讲，把它想象成一个非常大的字节数组
 - 实际上不是，但可以这样想
- 地址就像该数组中的索引
 - 并且，一个指针变量存储一个地址

■ 注意：系统为每个“进程”提供私有地址空间

- 将进程视为正在执行的程序
- 因此，程序可以修改自己的数据，但不能修改其他程序的数据

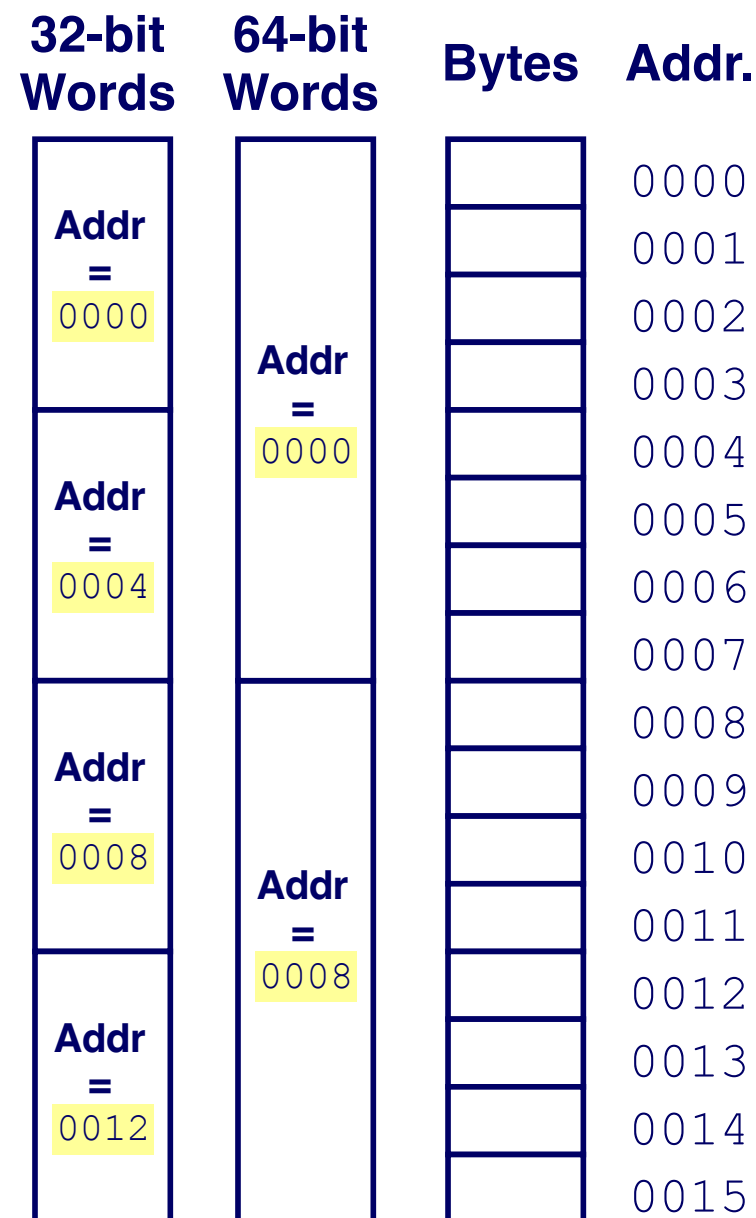
机器字

- **任何给定的计算机都有一个“字大小”**
 - 整数值数据的标称大小(nominal size)
 - 也是地址长度
 - 直到近些年，大多数机器使用 32 位（4 个字节）作为字长
 - 地址限制为 4GB(2^{32} 字节)
 - 越来越多的机器的字长为64 位
 - 可能有18 EB (exabytes) 的可寻址内存
 - 那是 18.4×10^{18}
 - 机器仍然支持多种数据格式
 - 字长的部分或倍数
 - 总是字节数的整数倍

面向字长的内存组织

■ 地址指定字节位置

- 字中第一个字节的地址
- 连续字的地址相差 4 (32 位) 或 8 (64 位)



数据表示示例

C Data Type	Typical 32-bit	Typical 64-bit	x86-64
<code>char</code>	1	1	1
<code>short</code>	2	2	2
<code>int</code>	4	4	4
<code>long</code>	4	8	8
<code>float</code>	4	4	4
<code>double</code>	8	8	8
<code>pointer</code>	4	8	8

字节排序

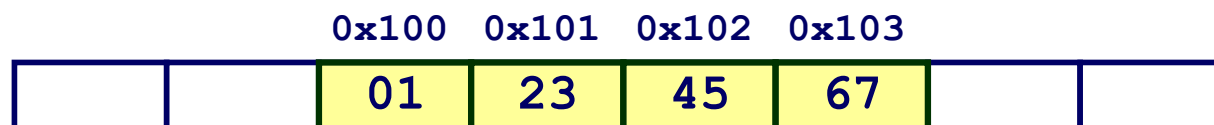
- 那么，多字节字中的字节在内存中是如何排序的呢？
- 约定
 - 大端法: Sun, PPC Mac, *Internet*
 - 最低有效字节具有最高地址
 - 小端法: *x86*, ARM processors running Android, iOS, and Windows
 - 最低有效字节具有最低地址

字节排序示例

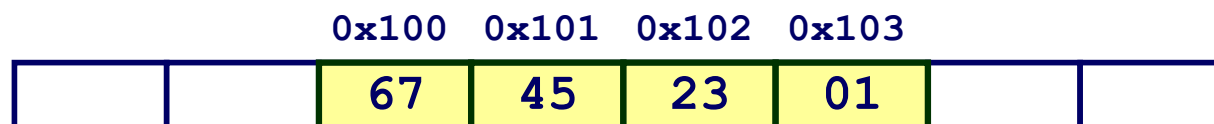
■ 示例

- 变量x 有四个字节值 0x01234567
- &x 表示的地址是 0x100

大端法



小端法



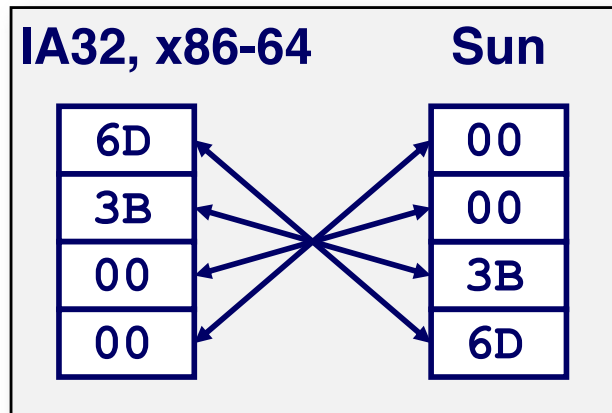
表示整数

Decimal: 15213

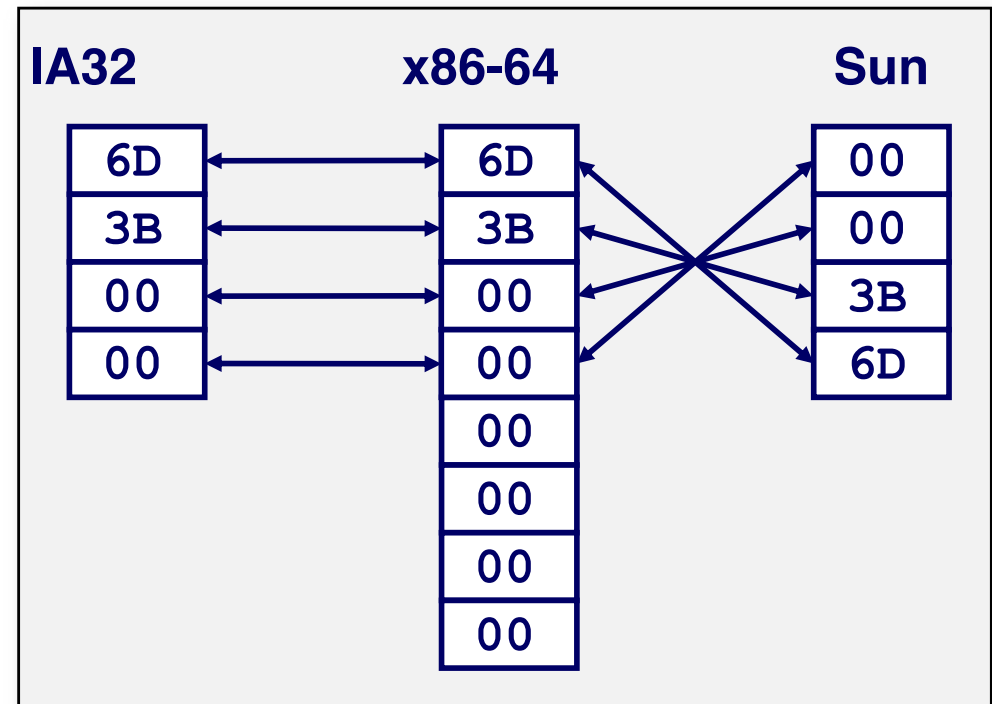
Binary: 0011 1011 0110 1101

Hex: 3 B 6 D

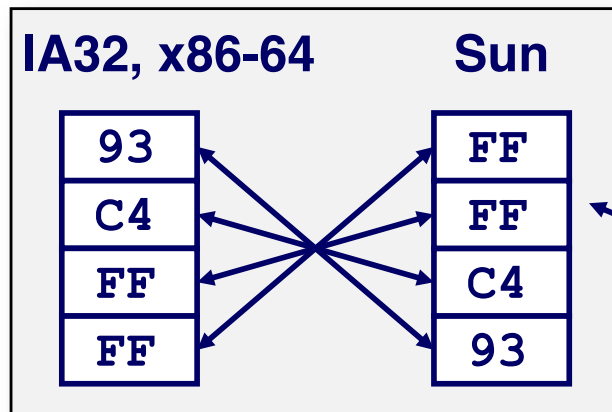
`int A = 15213;`



`long int C = 15213;`



`int B = -15213;`



补码表式

检查数据表示形式

■ 用于打印数据的字节表示的代码

- 将指针转换为 unsigned char * 允许作为字节数组处理

```
typedef unsigned char *pointer;

void show_bytes(pointer start, size_t len) {
    size_t i;
    for (i = 0; i < len; i++)
        printf("%p\t0x%.2x\n", start+i, start[i]);
    printf("\n");
}
```

Printf directives:

%p: Print pointer

%x: Print Hexadecimal

show_bytes 执行示例

```
int a = 15213;  
printf("int a = 15213;\n");  
show_bytes((pointer) &a, sizeof(int));
```

Result (Linux x86-64):

```
int a = 15213;  
0x7ffffb7f71dbc      6d  
0x7ffffb7f71dbd      3b  
0x7ffffb7f71dbe      00  
0x7ffffb7f71dbf      00
```

表示指针

```
int B = -15213;  
int *P = &B;
```

Sun	IA32	x86-64
EF	AC	3C
FF	28	1B
FB	F5	FE
2C	FF	82
		FD
		7F
		00
		00

不同的编译器和机器为对象分配不同的位置

每次运行程序甚至得到不同的结果

表示字符串

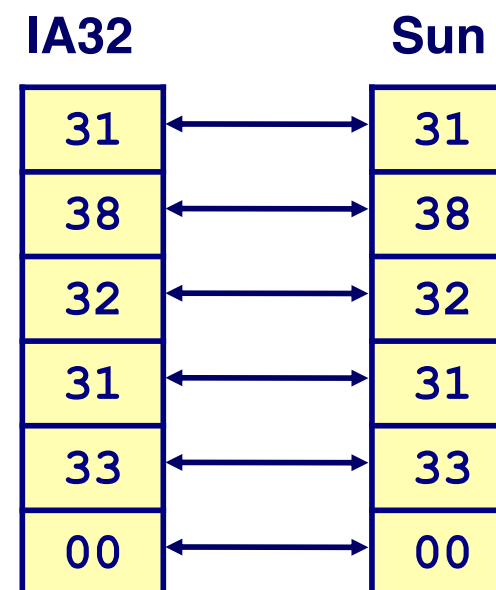
■ C语言中的字符串

- 用字符数组表示
- 每个字符以 ASCII 格式编码
 - 标准的 7 位字符集编码
 - 字符“0”的编码为 0x30
 - 数字 i 的编码为 $0x30+i$
- 字符串应该以空字符结尾
 - 最后一个字符 = 0

■ 兼容性

- 字节顺序不是问题

```
char S[6] = "18213";
```



读取字节反转列表

■ 拆解

- 二进制机器代码的文本表示
- 由读取机器代码的程序产生

■ 示例片段

Address	Instruction Code	Assembly Rendition
8048365:	5b	pop %ebx
8048366:	81 c3 ab 12 00 00	add \$0x12ab,%ebx
804836c:	83 bb 28 00 00 00 00	cmpl \$0x0,0x28(%ebx)

■ 破译数字

- 值：
0x12ab
- 填充到 32 位：
0x000012ab
- 拆分成字节：
00 00 12 ab
- 反转：
ab 12 00 00

THE END